CSE35101 Assignment 2 20211181 양성지

Used same method of starting the proxy server as in PA1's server.

```c
typedef struct cacher {        // will create cacher head; (SLL)
    char *url;
    char *response;
    int datasize;
    time_t expiration;
    struct cacher *next_cache;     // if null = end of cache
    struct cacher *before_cache;     // if null = end of cache
} cacher;

typedef struct {
    int fd;
    int opfd;
    char *buf;     // enough space
    int bufsize;
    int status;     // 0 = read all of client's request, 1 = parse
    int msg_read;
    char *method, *request, *protocol;
    char *hostname;
} data_center;
```

Structs used to save client's request, server's response.

```c
int curfd = events[i].data.fd;
switch (proxys[curfd].status)
{
// need to check inital line : GET
case 0: ;
```

Used switch + case to clarify the proxy's action.

```c
// just in case
if (proxys[curfd].msg_read + 1 == proxys[curfd].bufsize) {
    char *tmp_alloc = realloc(proxys[curfd].buf, proxys[curfd].bufsize * 2);
    if (tmp_alloc == NULL) {
        free(proxys[curfd].buf);
        epoll_ctl(epfd, EPOLL_CTL_DEL, curfd, NULL);
        close(curfd);
        continue;
    }
    proxys[curfd].buf = tmp_alloc;
    memset(proxys[curfd].buf + proxys[curfd].msg_read, 0, proxys[curfd].msg_read + 1);
    proxys[curfd].bufsize *= 2;
}

// more safe?
int read_bytes = read(curfd, proxys[curfd].buf + proxys[curfd].msg_read, proxys[curfd].bufsize - proxys[curfd].msg_read - 1);
```

In case of longer request, check and realloc the buffer if needed (same method is used when getting the server's response)

```c
// need to read more
if (strstr(proxys[curfd].buf, "\n\n") == NULL && strstr(proxys[curfd].buf, "\r\n\r\n") == NULL) break;
```

Check if header has all been written (being generous and support both ₩n and ₩r₩n)

```c
char *init_line = strtok_r(for_tok, "\r\n", &all_ptr);

// need same logic
char *method, *request, *protocol;
char *cut_char = " \t\r\n", *init_ptr;

int NULL_found = 0;

// check each part of input
method = strtok_r(init_line, cut_char, &init_ptr);    // GET
if (method == NULL) NULL_found += 1;

if (!NULL_found)
    request = strtok_r(NULL, cut_char, &init_ptr);          // link
if (request == NULL) NULL_found += 2;

if (!NULL_found)
    protocol = strtok_r(NULL, cut_char, &init_ptr);         // HTTP/1.0
if (protocol == NULL) NULL_found += 4;
```

Extract method, URL, protocol by tokenizing the string. Same is applied with Host header

as well.

```c
// ease of parsing (same start, if ':' comes first, things change if port NULL or not)
char *init_host, *init_port = NULL, *init_path = NULL;
init_host = proxys[curfd].request;
// has path (might not be default(/))
if ((init_path = strchr(proxys[curfd].request, '/')) != NULL) {
    *init_path = '\0';
    init_path++;
}
// has port (might not be default(80))
if ((init_port = strchr(proxys[curfd].request, ':')) != NULL) {
    *init_port = '\0';
    init_port++;
}
```

Extract actual host, port, path from URL.

```c
// method other than GET came in
if (strlen(proxys[curfd].method) != 3 || strncmp(proxys[curfd].method, "GET", 3) != 0) {
    free(proxys[curfd].buf); free(tmp_buf);
    // fprintf(stderr, "NOT GET\n");
    write(curfd, "HTTP/1.0 400 Bad Request\r\n\r\n", strlen("HTTP/1.0 400 Bad Request\r\n\r\n"));
    free(proxys[curfd].method); free(proxys[curfd].request); free(proxys[curfd].protocol);
    close(curfd);
    continue;
}
```

Error handling method, rest similar to this.

```c
int in_cache = 0;
cacher *c;
for (c = head->next_cache; c != NULL; c = c->next_cache) {
    // cache has same URL (host)
    if (strncmp(c->url, init_host, strlen(init_host)) == 0) {
        // need to check time (expire)
        if (c->expiration > time(NULL)) {
            // send data in the cache to the client
            in_cache++;
            break;
        }
        else {
            // need to write & check response's cache control & send msg to client
            // will just implement in a way to remove past cache, read all response, check cache-control, do as written
            c->before_cache->next_cache = c->next_cache;
            if (c->next_cache != NULL) c->next_cache->before_cache = c->before_cache;
            free(c->url); free(c->response);
            free(c);
            break;
        }
    }
}
```

Check cache, if in the cache, proxy will be able to know with in_cache

```c
// new setting on ev
ev.data.fd = serv_sockfd;
// level trigger, not edge trigger (EOLLIN | EPOLLET)
ev.events = EPOLLIN;
epoll_ctl(epfd, EPOLL_CTL_ADD, serv_sockfd, &ev);
proxys[serv_sockfd].fd = serv_sockfd;
proxys[serv_sockfd].status = 3;     // need to get ans from serv
proxys[serv_sockfd].msg_read = 0;
proxys[serv_sockfd].opfd = curfd;
proxys[serv_sockfd].buf = (char *)calloc(proxys[curfd].bufsize, sizeof (char));
proxys[serv_sockfd].bufsize = proxys[curfd].bufsize;
```

Create new fd to communicate with actual server.

```c
memcpy(write_buf, "GET /", 5);
offset += 5;
if (init_path != NULL) {
    memcpy(write_buf + offset, init_path, strlen(init_path));
    offset += strlen(init_path);
}

memcpy(write_buf + offset, " HTTP/1.0\r\nHost: ", strlen(" HTTP/1.0\r\nHost: "));
offset += strlen(" HTTP/1.0\r\nHost: ");
if (head_host != NULL) {
    memcpy(write_buf + offset, head_host, strlen(head_host));
    offset += strlen(head_host);
}
write_buf[offset++] = '\r';
write_buf[offset++] = '\n';
```

Organizing message to write to server.

```
### Testing: http://www.kaist.edu
http://www.kaist.edu: [PASSED] (5/5 points)

### Testing: http://info.in2p3.fr/
http://info.in2p3.fr/: [PASSED] (5/5 points)

### Testing: http://info.cern.ch/
http://info.cern.ch/: [PASSED] (5/5 points)

### Testing: http://info.cern.ch/hypertext/WWW/TheProject.html
http://info.cern.ch/hypertext/WWW/TheProject.html: [PASSED] (5/5 points)

##############################

Extended Functionality Test

##############################

### Testing error case #1:
Request without host header field: [PASSED] (8/8 points)

### Testing error case #2:
Request with method other than GET (case 1): [PASSED] (2/2 points)
Request with method other than GET (case 2): [PASSED] (2/2 points)
Request with method other than GET (case 3): [PASSED] (2/2 points)
Request with method other than GET (case 4): [PASSED] (2/2 points)

### Testing error case #3:
Request with different HTTP version (other than v1.0): [PASSED] (8/8 points)

### Testing error case #4:
Invalid host header: [PASSED] (8/8 points)

### Testing error case #5:
Different URI and host: [PASSED] (8/8 points)

Total Score:    60 / 60
```

```
### Testing: http://www.kaist.edu
http://www.kaist.edu: [PASSED] (5/5 points)

### Testing: http://info.in2p3.fr/
Proxy: date: Wed, 01 Oct 2025 12:33:47 GMT
Direct: date: Wed, 01 Oct 2025 12:33:48 GMT
http://info.in2p3.fr/: [FAILED] (0/5 points)

### Testing: http://info.cern.ch/
http://info.cern.ch/: [PASSED] (5/5 points)

### Testing: http://info.cern.ch/hypertext/WWW/TheProject.html
http://info.cern.ch/hypertext/WWW/TheProject.html: [PASSED] (5/5 points)

##############################

Extended Functionality Test

##############################

### Testing error case #1:
Request without host header field: [PASSED] (8/8 points)

### Testing error case #2:
Request with method other than GET (case 1): [PASSED] (2/2 points)
Request with method other than GET (case 2): [PASSED] (2/2 points)
Request with method other than GET (case 3): [PASSED] (2/2 points)
Request with method other than GET (case 4): [PASSED] (2/2 points)

### Testing error case #3:
Request with different HTTP version (other than v1.0): [PASSED] (8/8 points)

### Testing error case #4:
Invalid host header: [PASSED] (8/8 points)

### Testing error case #5:
Different URI and host: [PASSED] (8/8 points)

Total Score:    55 / 60
```

Like this, the second testcase's result vary (same as last Q&A in Error Message Handling Issues)

```c
char *pp = strstr(cache_header, "private");
// if pp != NULL, can't cache = just clean & exit
if (pp == NULL) {
    // pp = strstr(cache_header, "public");
    char *ap = strstr(cache_header, "max-age=");
    if (ap != NULL) {
        // caching
        cacher *new_cache = (cacher *)calloc(1, sizeof (cacher));

        new_cache->url = malloc(strlen(proxys[opfd].hostname) + 1);
        memcpy(new_cache->url, proxys[opfd].hostname, strlen(proxys[opfd].hostname) + 1);
        new_cache->response = (char *)calloc(proxys[curfd].bufsize, sizeof (char));
        memcpy(new_cache->response, proxys[curfd].buf, proxys[curfd].msg_read + 1);

        new_cache->datasize = proxys[curfd].msg_read;
        new_cache->expiration = time(NULL) + atoi(ap + 8);

        if (head->next_cache != NULL) head->next_cache->before_cache = new_cache;
        new_cache->next_cache = head->next_cache;
        head->next_cache = new_cache;
        new_cache->before_cache = head;
```

Verify and cache server's response.

references :

https://www.ietf.org/rfc/rfc1945.txt

https://learn.microsoft.com/ko-kr/windows/win32/api/winsock2/nf-winsock2-listen

https://en.cppreference.com/w/c/string/byte.html

https://stackoverflow.com/questions/19338846/winsock2-getaddrinfo-to-resolve-a-hostname

https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_URL

https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages